Архитектура ИИ-агента для анализа причин аномалий качества данных в DB Monitoring

Домашнее задание B14 «Генеративный ИИ и ИИ-агенты», слайд 49; Issue #43. **Статус: проектное предложение.** Агент не реализован и не входит в DB Monitoring; программная реализация заданием не требуется. Существующий `POST /api/explain` — один вызов LLM по заранее собранному контексту с шаблонным ответом при сбое, а не агент (аудит ограничений). Описание опирается на текущую архитектуру и кейс.

1. Назначение и сценарий использования

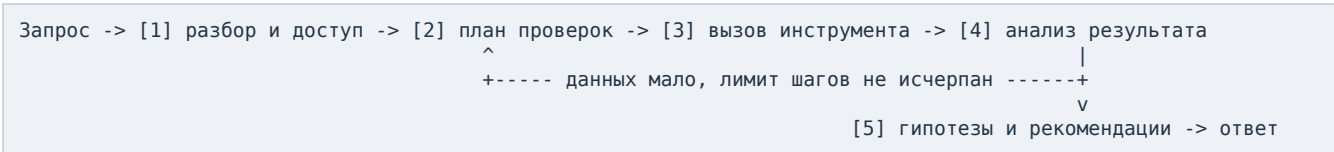
Пользователь — специалист по качеству данных. Детектор DB Monitoring или дашборд показали рост доли NULL, и специалист спрашивает: «Почему в таблице `orders` за последние сутки выросла доля NULL?». Сейчас он вручную сопоставляет графики `null_rate` и `row_count`, NULL по колонкам, изменения схемы и сигналы детекторов. Агент выполняет эти проверки сам в рамках текущего проекта пользователя и возвращает отчёт:

- **факты:** когда начался рост, на сколько изменилась доля NULL, какие колонки дали прирост, были ли изменения схемы и скачок числа строк — с временем наблюдения и названием инструмента-источника;
- **гипотезы:** 1–3 возможные причины, например «добавлена nullable-колонка без значений по умолчанию» или «поступила партия строк с незаполненным полем», с подтверждающими и противоречащими фактами;
- **рекомендации:** что проверить вручную (журнал ETL, источник загрузки, миграцию) и каких данных агенту не хватило.

Ответ явно помечается как гипотеза, требующая проверки человеком.

2. Пайплайн обработки запроса

Агент работает по циклу «рассуждение → действие → наблюдение» (паттерн ReAct) с ограничением числа шагов.



1. **Получение запроса.** Пользователь и проект берутся из авторизованной сессии, а не из текста запроса. LLM извлекает имя таблицы, метрику и период; инструмент `resolve_table` подтверждает, что таблица есть в текущем проекте. При неоднозначности агент задаёт уточняющий вопрос и не продолжает.
2. **Определение необходимых проверок.** План строится из фиксированного перечня: подтвердить рост и момент его начала; найти колонки с приростом NULL; проверить изменения схемы; сравнить динамику числа строк; учесть сигналы детекторов. LLM может менять порядок и пропускать проверки, но не добавлять операции вне перечня.
3. **Вызов инструментов.** Аргументы проверяются по схеме инструмента, `project_id` подставляется из сессии. Все инструменты только читают агрегированные данные хранилища метрик; результат сокращается до значимых чисел.
4. **Анализ результатов.** Разности, отношения и сравнение «до/после» считает код, LLM интерпретирует готовые числа. После каждого наблюдения агент решает: данных достаточно или нужна следующая проверка. Цикл завершается при достаточных фактах, исчерпании лимита (например, 8 вызовов) или ошибке инструмента.
5. **Формирование гипотезы и рекомендации.** Каждая гипотеза ссылается на идентификаторы наблюдений из памяти; утверждения без ссылки отбрасываются или помечаются как неподтверждённые. Если LLM недоступна, формируется шаблонный отчёт из собранных фактов.

3. Инструменты агента

Все инструменты — **предлагаемые**: в DB Monitoring их нет. Справа указаны существующие функции хранилища и маршруты API, на которые они могли бы опираться.

Предлагаемый инструмент	Назначение (входы: таблица, период)	Возможная основа в текущем коде
`resolve_table`	Найти таблицу в текущем проекте и подтвердить доступ к ней	`GET /api/tables`
`get_quality_history`	Ряды `null_rate` и `row_count`: момент начала и величина роста, изменение объёма	`get_metrics`, `GET /api/metrics/<table>`
`get_column_nulls`	NULL по колонкам до и после начала роста: какие колонки дали прирост	`get_latest_null_counts` даёт только последний сбор; чтение на заданный момент потребует доработки
`get_schema`	Текущая схема: имена, типы, признак nullable	`get_schema_snapshot`, `GET /api/schema/<table>`
`get_schema_changes`	История изменений схемы: добавленные, удалённые, изменённые колонки	`get_schema_events`, `GET /api/schema/<table>/changes`
`get_detector_signals`	Оценки Isolation Forest, точки изменения и дрейф как дополнительный контекст	`get_anomaly_scores`, `get_changepoints`, `get_drift_report`

Журналы ETL и историю SQL DB Monitoring не собирает, поэтому агент называет их в рекомендациях как недостающие источники. Инструментов записи, выполнения SQL и чтения содержимого строк нет.

4. Организация памяти

Используется только **кратковременная память в рамках одного запроса**: структурированное рабочее состояние в процессе обработки, а не просто история диалога.

```
request:      текст вопроса, пользователь, project_id, время запроса
target:       таблица, метрика null_rate, период анализа
plan:         [{check: "column_nulls", status: done | pending | skipped | error}, ...]
observations: [{id: "o3", tool, args, summary: "email: +1 840 NULL", data_ts}, ...]
findings:     [{text: "рост совпал с добавлением колонки email", evidence: ["o3", "o4"]}]
budget:       шаги, токены, ошибки инструментов
```

Передача контекста между шагами. Перед каждым шагом промпт собирается из состояния: инструкции агента, исходный вопрос, выбранная таблица, план со статусами и краткие сводки наблюдений. Полные результаты остаются в состоянии для проверки итоговых ссылок кодом, поэтому контекст LLM не растёт с каждым вызовом. Имена таблиц и колонок передаются отдельным блоком данных: они недоверенные и не должны восприниматься как инструкции. После ответа состояние удаляется; для аудита достаточно сохранить отчёт и список вызовов. Постоянная память (например, база причин прошлых инцидентов) в предложение не входит.

5. Границы предлагаемого решения

- **Не реализованная функциональность.** Документ описывает проект агента; в коде DB Monitoring агента, перечисленных инструментов и цикла нет.
- **Только анализ и рекомендации.** Агент не исправляет данные, не выполняет SQL и миграции, не меняет настройки, модели и пороги. Рекомендации — текст; решение принимает и применяет специалист.
- **Гипотеза, а не установленная причина.** Агент видит только агрегированные метрики, схему и сигналы детекторов; без журналов ETL причину подтвердить нельзя. Уверенность LLM не откалибрована.
- **Детектор не заменяется.** Решение об аномалии по-прежнему принимает classical ML; агент помогает разобрать уже замеченное отклонение.
- **Безопасность при возможной реализации** — как для LLM-объяснений: доступ проверяется до чтения данных, изоляция по проекту и источнику, минимум метаданных для внешней LLM, защита от инструкций в именах колонок.